How Training Builds a Reasoning Procedure

Soroush Esfahanian

Which computational dependencies must become functional for a reasoning procedure to work, and how do different forms of supervision establish them? I propose to connect changes in reasoning during training to the internal operations that produce them. Starting with reasoning under temporary assumptions, I would compare supervised fine-tuning and reinforcement learning, identify how candidate operations become causally connected, and test whether those explanations guide more efficient learning. The aim is a developmental account that predicts errors and useful training interventions, with explicit limits on where it generalizes.

Motivation and research gap

A model might detect a contradiction yet fail to identify which temporary assumption it invalidates. Improving answer accuracy does not reveal whether training supplied a missing operation, improved its selection, or connected it to the rest of a procedure. Distinguishing these possibilities could explain why apparently successful training produces brittle reasoning and indicate what supervision is needed next.

Mechanistic studies have traced algorithm formation during grokking [1], compared SFT and RL features across checkpoints [2], and separated mechanisms that induce reasoning behaviors from heuristics that select them [3]. Logical circuit analysis identifies modular computations [4], while mechanistic data attribution links training examples to the emergence of interpretable units [5]. Building on these precedents, I would investigate the acquisition of dependencies between operations within a defined procedure, and whether this account predicts which changes to training improve its use on unfamiliar problems. This requires more than detecting feature changes or naming strategies in generated text.

Research questions

First, when do representations of assumptions, their consequences, and inconsistency begin to influence one another appropriately, and how does their acquisition depend on supervision? Second, can a diagnosis of a missing or misused dependency predict which targeted training change improves generalization with fewer examples or less computation? A computational dependency means that intervening on one candidate operation changes a downstream operation in the way the proposed procedure predicts.

Proposed approach

Establish a controlled learning setting. I would build on programmable deductive reasoning tasks [6], initially using short proofs under temporary assumptions, with contradiction as a leading case. Premises would be consistent, with hypothetical branches explicitly scoped. Problems would vary assumption scope, inference depth, and irrelevant facts; a symbolic verifier would establish validity. Matched examples would distinguish a contradiction caused by the active assumption from one in an unrelated branch. Holding out rule combinations and deeper proofs would test reuse beyond familiar templates. The proof decomposition supplies competing hypotheses about computation, not an assumption that the model follows a human algorithm.

Compare acquisition routes. Starting from one open model of approximately 1-3 billion parameters, I would compare SFT on verified proofs, RL with verifiable outcome rewards, and an SFT-to-RL sequence across repeated seeds and saved checkpoints. The initial runs would share a problem distribution and initialization. Demonstrations and scalar rewards convey different information, so additional controls would vary access to proof traces within a fixed training recipe. I would compare both performance-matched checkpoints and learning curves against examples and training compute, rather than attribute every difference to the training algorithm alone.

Test internal dependencies longitudinally. Probes would locate candidate representations, followed by activation and path interventions between minimally different examples. For instance, replacing a representation of an assumption's scope should change which contradiction licenses its rejection, while preserving unrelated deductions. Single and joint interventions, disruption and rescue tests, and naturally generated versus standardized prefixes would distinguish genuine dependencies from answer encoding or changes in wording. Causal abstraction [7] provides a way to specify these predictions. Interventions would

first be validated within each checkpoint; functional roles would be compared across checkpoints without assuming that neuron identities remain stable.

Keep the interpretation falsifiable. Highly expressive alignment maps can manufacture misleading algorithmic interpretations [8]. I would therefore constrain map complexity, test predictions on unseen problem structures, and include unrelated-state and random-subspace controls. A probe's success alone would not establish use, and a failed sparse-circuit search would not establish the absence of a distributed mechanism. The intended result is a validated partial account, including alternative routes and its failure conditions.

Use the explanation to improve learning. At a checkpoint with a diagnosed dependency failure, I would branch training into a targeted curriculum, random additional training, and difficulty-matched behavioral error correction. If scope tracking is intact but contradiction attribution fails, the targeted examples would vary only the connection between an assumption and its consequences. Success would require the predicted internal change alongside improved performance on held-out compositions. I would report training cost to a fixed generalization level, including the cost of obtaining the diagnosis. This tests a causal claim about learning separately from the inference-time interventions.

Feasibility and expected contribution

The first three months would establish the task generator, baselines, and intervention controls. Months four to eight would develop and challenge one dependency account; months nine to twelve would test the resulting training intervention and, if feasible, replicate it in a second model family. Smaller transformers would provide a fallback for validating the analysis before scaling.

The contribution would be a reusable framework for studying the development of reasoning procedures, a bounded explanation of how supervision establishes computational dependencies, and evidence about whether that explanation improves training efficiency. Different acquisition routes would be informative if they predict distinct errors or training needs; universal developmental stages are not required. Later work could extend the account to other procedures and compare learning order and characteristic errors with human behavior. Neural comparison would follow a specific computational hypothesis and collaboration with cognitive neuroscientists.

Selected references

[1] Nanda et al. (2023). *Progress measures for grokking via mechanistic interpretability*. ICLR.

[2] Shi et al. (2026). *Why Does Reinforcement Learning Generalize? A Feature-Level Mechanistic Study of Post-Training in Large Language Models*. ACL.

[3] Venhoff et al. (2026). *Base Models Know How to Reason, Thinking Models Learn When*. ICML.

[4] Hong et al. (2025). *A Implies B: Circuit Analysis in LLMs for Propositional Logical Reasoning*. NeurIPS.

[5] Chen, Luo, and Pan (2026). *Mechanistic Data Attribution: Tracing the Training Origins of Interpretable LLM Units*. ICML.

[6] Saparov et al. (2023). *Testing the General Deductive Reasoning Capacity of Large Language Models Using OOD Examples*. NeurIPS.

[7] Geiger et al. (2025). *Causal Abstraction: A Theoretical Foundation for Mechanistic Interpretability*. JMLR 26(83).

[8] Sutter et al. (2025). *The Non-Linear Representation Dilemma: Is Causal Abstraction Enough for Mechanistic Interpretability?* NeurIPS.